

Scope, Related Work, and Positioning

This section situates the exemplar scientifically and states explicit boundaries. The goal is not to compete with monographs on nonlinear programming (Nocedal and Wright 2006; Bertsekas 1999), but to show how a minimal, test-backed optimization story fits the template's reproducibility and rendering stack (Peng 2011).

Smooth unconstrained minimization via first-order updates has a long lineage, from Cauchy's early descent perspective (Cauchy 1847) to modern treatments of convex problems (Boyd and Vandenberghe 2004) and accelerated gradient schemes (Nesterov 2013). Polyak's classical discussion of gradient convergence factors remains relevant when interpreting empirical iteration counts (Polyak 1964). The present manuscript restricts attention to **fixed-step** gradient descent on a **convex quadratic**, where rates reduce to scalar linear recurrences in the error (sec. sec:results, eq. ??).

Practical machine-learning optimizers (e.g., Adam (Kingma and Ba 2015)) introduce momentum, adaptive preconditioning, or noise from minibatching. Those methods are **out of scope** for `template_code_project`: the exemplar deliberately keeps the algorithm minimal so that failures (divergent α , iteration caps) are interpretable without confounding from stochastic sampling or line-search logic.

What this project proves about the template I

The scientific claims through `sec. sec:introduction`, `sec. sec:methodology`, and `sec. sec:results` are standard textbook material. The **non-standard** contribution is procedural: configuration in `manuscript/config.yaml` drives `run_convergence_experiment()`, figures, CSV exports, and `RESULT_*` token substitution (`scripts/z_generate_manuscript_variables.py`) so that PDF, HTML, and validation logs refer to the same numbers. That pattern is what downstream projects should copy—whether the domain is optimization, differential equations, or Bayesian workflows.

Explicit limitations I

Explicit limitations I

1. **Dimensionality:** Default experiments emphasize $d = 1$ with $A = I$ for transparent plotting; the benchmark figure explores $d > 1$ only with identity Hessians, so no ill-conditioning effects appear.
2. **Step-size policy:** Only constant α is implemented in `src/optimizer.py`; there is no Wolfe or Armijo backtracking.
3. **Global optimization:** Convexity is assumed; no basin-hopping or restarts are studied.
4. **Numerical model:** Double-precision floating point only; no interval or arbitrary-precision analysis.

Explicit limitations I

These limitations are intentional: they narrow the failure surface so that infrastructure concerns—tests, logging, figure registration, and PDF cross-references—remain visible rather than buried under algorithmic complexity.